

HClave: An isolated execution environment design for hypervisor runtime security

Qihang Zhou^a, Wenzhuo Cao^{a,b}, Xiaoqi Jia^{a,b,*}, Shengzhi Zhang^c, Jiayun Chen^{a,b},
Nan Jiang^{a,b}, Weijuan Zhang^a, Haichao Du^a, Zhenyu Song^a, Qingjia Huang^a

^a Institute of Information Engineering, Chinese Academy of Sciences, China

^b School of Cyber Security, University of Chinese Academy of Sciences, China

^c Metropolitan College, Boston University, USA

ARTICLE INFO

Keywords:

Virtualization security
Hypervisor
Memory isolation
Secure cloud component development

ABSTRACT

Virtualization is the cornerstone of cloud computing, but the hypervisor, the crucial software component that enables virtualization, is known to suffer from various attacks. It is challenging to secure the hypervisor due to at least two reasons. On one hand, commercial hypervisors are usually integrated into a privileged Operating System (OS), which brings in a larger attack surface. On the other hand, multiple Virtual Machines (VM) share a single hypervisor, thus a malicious VM could leverage the hypervisor as a bridge to launch “cross-VM” attacks. In this work, we propose HClave, an isolated execution environment (IEE) design for hypervisor runtime. We decouple the virtualization layer into a tiny trusted computing base (TCB), a large non-secure OS, and multiple HClave IEEs through a bidirectional isolation approach. HClave extends the nested kernel approach to deprive the traditional OS from accessing the tiny TCB’s memory and creates an IEE for hypervisor runtime. We implemented HClave based on KVM and evaluated its effectiveness and efficiency through case studies. Experimental results show that HClave can significantly improve the security of the hypervisor with reasonable runtime overhead.

1. Introduction

The hypervisor, a crucial software component in virtualization, provides resource management and facilitates the operation of multiple Virtual Machines (VMs) on a single physical server. However, modern hypervisors are susceptible to severe security threats from two main aspects. Firstly, the hypervisor processes untrusted VM states as input, such as fetching VM instructions for interpretation and execution during instruction emulation. For example, a VM escape vulnerability could be exploited by malicious VMs to execute arbitrary code within the hypervisor, thereby jeopardizing the entire virtualization platform. Furthermore, the monolithic design used to control multiple VMs intensifies the issue of cross-VM attacks via the compromised hypervisor. Secondly, a traditional hypervisor often incorporates a more complex, yet vulnerable, privileged Operating System (OS) (Kivity et al., 2007) to aid in virtualization. Attackers can exploit the vulnerabilities in the OS kernel, swiftly gain control of the hypervisor, and subsequently compromise the confidentiality and integrity of all the VMs.

In recent years, the Isolated Execution Environment (IEE) has emerged as a new computing paradigm, gaining popularity due to its reliable privacy protection. For example, the Trusted Execution Environment (TEE), a typical example of IEEs, incorporates a hardware-enforced memory region where sensitive code can be securely executed. Many IEE-based approaches have been proposed by researchers to enhance hypervisor security. These approaches can be broadly classified into two categories: Isolation-based IEE (Xia et al., 2022; Li et al., 2019, 2022) and Encryption-based IEE (Intel, 2020; Intel Corporation, 2016; Kaplan et al., 2016; Wu et al., 2018). The former provides a hardware-isolated memory area for secure execution of target code, while the latter ensures the security of sensitive data by encrypting the memory of VMs. However, these approaches encounter several limitations. Firstly, some of them, such as (Intel Corporation, 2016), are unable to protect complex, monolithic system software like hypervisors due to limited memory space. Secondly, the majority of these works

* Corresponding author at: Institute of Information Engineering, Chinese Academy of Sciences, No.19, Shucun Road, Haidian District, Beijing, China.

E-mail addresses: zhouqihang@iie.ac.cn (Q. Zhou), caowenzhuo@iie.ac.cn (W. Cao), jiaxiaqi@iie.ac.cn (X. Jia), shengzhi@bu.edu (S. Zhang), chenjiayun@iie.ac.cn (J. Chen), jiangnan@iie.ac.cn (N. Jiang), zhangweijuan@iie.ac.cn (W. Zhang), duhaichao@iie.ac.cn (H. Du), songzhenyu@iie.ac.cn (Z. Song), huangqingjia@iie.ac.cn (Q. Huang).

¹ This paper is an extended version of the conference paper published in IPDPS’22: SecFortress: Securing Hypervisor using Cross-layer Isolation (Zhou et al., 2022).

are dependent on specific hardware-secure platforms, for instance, Intel Corporation (2016), Li et al. (2022), T. Alves (2004), resulting in a lack of scalability. Lastly, certain works, like (Wu et al., 2018), necessitate instrumentation within the guest OS, which is not only incompatible with commodity Guest OS but also vulnerable to malicious VMs.

In response to the aforementioned challenges, we propose a novel software IEE design for hypervisor runtime that does not rely on specific hardware-secure mechanisms or modifications to the guest. We introduce an instance of the IEE, named HClave, which holds the same privileges as the host OS and hypervisor, yet neither of them is part of our system's Trusted Computing Base (TCB). We construct a compact secure monitor, serving as the TCB for the entire system, through a tamper-proof protection mechanism at the same privilege level. This secure monitor offers three types of security services: memory protection, instruction protection, and context-switching protection, all of which ensure the confidentiality and integrity of each HClave. During its runtime, each HClave is granted access to these services. Moreover, we enforce control-flow and data-flow isolation among different HClaves and safeguard each HClave from untrusted system software. We also furnish three types of interfaces for managing the lifecycle of HClave.

We have implemented a prototype based on Linux-KVM, incorporating minor alterations to the Linux kernel on both the ARM64 and x86 platforms. The TCB of our system is approximately 3,000 lines of code (LoC). To illustrate the security advantages of our system, we conducted extensive analyses using various case studies. We scrutinized 22 hypervisor-related vulnerabilities in-depth, which included threats from malicious virtual machines (VMs) and untrusted system software. We then assessed HClave's performance against these corresponding exploits. Our findings revealed that the majority of these vulnerabilities resulted in memory problems. The primary cause of these issues was the hypervisor's failure to enforce appropriate memory isolation among different VMs. Our evaluation results indicate that HClave can effectively counter these exploits with a reasonable impact on performance.

Contributions. The contribution of this paper is summarized below:

- We propose a novel Isolated Execution Environment (IEE) methodology for hypervisor runtime. Each IEE instance, named HClave, is isolated from the others and the untrusted system software. We enhance the hypervisor runtime security by confining untrusted VMs into a per-VM HClave.
- We minimize the TCB of the virtualization software stack by moving both the system software and HClave out of the TCB and limiting their respective capabilities.
- We implemented our system based on KVM and evaluated it comprehensively using 22 real-world hypervisor vulnerabilities. Our evaluation results show that HClave can defeat the above exploits with reasonable overhead.

2. Background and motivation

2.1. Hypervisors

The hypervisor, a vital component of the virtualization software stack, can be categorized into two types: bare-metal hypervisors and hosted hypervisors. This paper primarily focuses on hosted hypervisors, which are typically integrated into traditional OSes responsible for managing physical resources. Hosted hypervisors, loaded as kernel modules, support virtualization. They typically access physical resources by invoking the host OS to facilitate CPU, memory, and I/O virtualization. For instance, KVM is a hosted hypervisor that is based on the Linux OS and requires hardware-assisted virtualization support. The Linux OS retains the responsibility of managing physical resources, hence KVM implements CPU and memory virtualization by calling specific Linux kernel functions.

2.2. Isolated execution environment

An Isolated Execution Environment (IEE) is a secure, isolated environment that ensures internal processes are not disrupted by external factors, while also preventing internal processes from directly compromising external systems. These technologies can be grouped into three categories based on the entity operating within: process-related IEEs (e.g., Haven Baumann et al., 2014, InkTag Hofmann et al., 2013), OS-related IEEs (e.g., ARM TrustZone T. Alves, 2004), and VM-related IEEs (e.g., Intel TDX Intel, 2020, AMD SEV Kaplan et al., 2016, DeHype Wu et al., 2013). Memory isolation is a crucial security mechanism provided by IEEs, effectively safeguarding the confidentiality and integrity of internal code and data from untrusted systems. The memory isolation mechanisms of IEEs can be divided into two types. The first is physical memory isolation, such as (T. Alves, 2004), which segregates physical memory areas for the execution of sensitive processes. The second type is encryption, such as (Kaplan et al., 2016), which ensures data confidentiality but does not guarantee integrity.

2.3. Nested kernel

The nested kernel approach (D et al., 2015) incorporates a compact isolated kernel within a monolithic kernel. By intercepting all updates to the virtual address translation, the nested kernel safeguards physical memory and significantly reduces the Trusted Computing Base (TCB) in a complex system. Additionally, the nested kernel proposes a novel Memory Management Unit (MMU) virtualization approach to isolate the TCB from untrusted components, differing from traditional approaches that utilize different CR3 base addresses or CPU privilege levels. The approach using different CR3 base addresses isolates various contexts by creating multiple pagetables. The transition between isolated contexts is achieved by switching the CR3 base address. However, this approach necessitates frequent Translation Lookaside Buffer (TLB) flushes, leading to substantial performance overhead. Another approach isolates contexts using the CPU privilege mechanism, such as nested virtualization (Ben-Yehuda et al., 2010; Zhang et al., 2011), with the TCB typically operating at a higher privilege level than untrusted components. This approach also incurs significant performance overhead due to context switches at different privilege levels. Unlike the aforementioned approaches, the nested kernel monitors all modifications to the virtual-to-physical mapping and eliminates all sensitive instructions from the untrusted components. All memory operations in the untrusted components are restricted by the isolated kernel. Consequently, the nested kernel achieves logical isolation between the isolated kernel and untrusted components within the same address space using a single privilege level.

2.4. Motivation

Hosted hypervisors, typically integrated into system software, primarily face two types of threats: those originating from the host mode and those from the guest mode. On one hand, the host mode software primarily comprises a hypervisor, system software, and numerous host applications. Given that the system software and the hypervisor often share the same address space, the system software can freely access the hypervisor's memory. The Common Vulnerabilities and Exposures (CVE) (CVE, 2021) further reveals that the system software harbors more vulnerabilities than the hypervisor, rendering it an ideal 'bridge' for launching attacks on the latter. Consequently, isolating the system software from the hypervisor and VMs emerges as an effective strategy to minimize the attack surface. For example, HypSec (Li et al., 2019), based on the microkernel design and ARM VE (Azab et al., 2014), eliminates the host OS and most of the hypervisor's functions from the TCB, subsequently depriving them. Solutions like Haven (Baumann et al., 2014) and S-NFV (Shih et al., 2016) employ Intel SGX to safeguard

Table 1
Attacks considered and not considered.

Malicious role \ Attack	Read or write to VM/HClave	Read or write to host OS	DoS to VM/HClave	DoS to host OS
Current VM	Not considered	Considered	Not considered	Partial Considered
Current VM's HClave	Considered	Considered	Not considered	Not Considered
other VM	Considered	Considered	Considered	Partial Considered
other VM's HClave	Considered	Considered	Considered	Not Considered
Host OS	Considered	Not considered	Not considered	Not considered

sensitive data against a malicious OS and hypervisor. However, due to memory constraints, SGX is not suitable for protecting large software.

On the other hand, modern hypervisors utilize a multitude of uncertified VM statuses as input. This opens up the possibility for malicious VMs to exploit vulnerabilities, compromise the hypervisor, and subsequently seize control of the entire virtualization platform. In addition to the programming vulnerabilities, the issue of a single point of failure also poses a significant problem. The fact that multiple VMs often share a single hypervisor implies that if a malicious VM gains control of the hypervisor, it can inflict damage on all other VMs. Furthermore, a malicious VM can execute a DoS attack on other VMs or the hypervisor, leading to a system-wide crash or freeze. Solutions like HyperLock (Wang et al., 2012) and DeHype (Wu et al., 2013) create a KVM instance for each VM to confine the impact to a per-VM instance. Intel TDX (Intel, 2020) isolates virtual machines from the hypervisor and any non-TD software on the platforms by incorporating a hardware-enforced secure-arbitration module. However, these solutions lack specific protection for hypervisors and require special hardware support.

To address the above problems, we believe it is necessary to design a reliable IEE, which not only isolates the system software from the hypervisor but also limits the privileges of each hypervisor instance. Such an IEE mechanism should effectively address both of those threats discussed above without requiring specific secure hardware, to ensure its scalability and portability.

3. HClave methodology

3.1. Threat model

In our assumptions, we consider the scenario where an attacker can exploit vulnerabilities in the system software to gain control and manipulate the HClave (hypervisor runtime instance). This control manipulation can lead to compromising the confidentiality and integrity of the VMs. Additionally, the attacker may intentionally deploy a malicious VM to launch attacks on the HClave or the system software through privilege escalation, thereby compromising other HClaves or VMs. Furthermore, the attacker can exploit vulnerabilities within a normal VM to gain full control over an application and attempt to attack the HClave. In addition, we assume that the bootup security of the virtualization platform is guaranteed by the secure boot (Tboot, 2021).

A VM deliberately leaking its sensitive data, side-channel attacks, host DoS attacks, and physical attacks are beyond the scope of this paper. Existing defense solutions against side-channel attacks, e.g., Chartre and Marsden (2018), can be integrated into our system. Table 1 shows the complete threat model matrix of HClave.

3.2. TCB design of HClave

Fig. 1 presents the architecture of our system. We partition the virtualization layer into a secure monitor, multiple HClaves, and a deprived OS. Our system minimizes the TCB of the virtualization platform by including the hardware and the secure monitor (around 2K LOC in total) only, which makes formal verification possible. The secure monitor lies in parallel with the deprived OS and HClaves

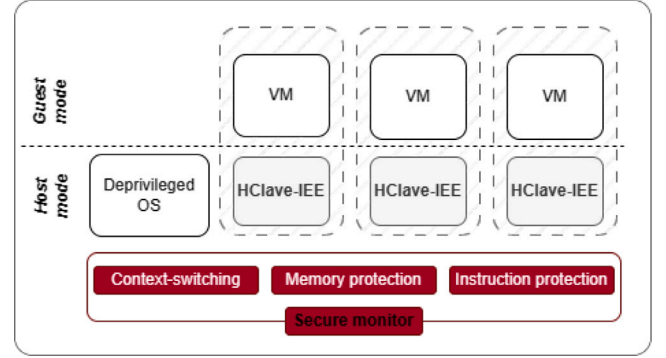


Fig. 1. System overview.

but is isolated from them via a virtual MMU memory protection mechanism. Meanwhile, we design an isolated execution environment (IEE), named HClave, through the CPU paging mechanism with three types of security services from the secure monitor, so the deprived OS cannot access the memory used by HClave nor tamper with its control-flow. In addition, each VM has its dedicated HClave, isolated from each other, which is enforced by the secure monitor. All HClaves share the same code, but each has its private data. To mitigate the single point of failure problem, we implement a security isolation mechanism to confine the control-flow and data-flow of HClave instances, and a series of policies to confine their memory access.

3.2.1. Tamper-proof protection

The secure monitor serves as the most crucial and foundational element of HClave protection. If the secure monitor becomes compromised, it invalidates all security mechanisms within the system. Therefore, it is imperative to ensure its tamper-proof nature. A common strategy involves abstracting an additional layer to operate the secure monitor. In software-based solutions, nested virtualization (Ben-Yehuda et al., 2010; Zhang et al., 2011) is a commonly used method, but it introduces significant performance overhead. To run the secure monitor securely and efficiently, we extend the nested MMU, which establishes a memory protection domain by virtualizing the MMU. This allows for the monitoring of all memory mapping updates within a large-scale system at an equivalent privilege level. Additionally, the nested MMU explicitly eliminates all instructions that could potentially modify the MMU from non-TCB components.

We achieve MMU virtualization by setting all kernel page-table-pages to read-only during the system initialization phase and activating the CPU's write-protection (WP) mechanism. Whenever the deprived OS attempts to update kernel page-table-pages, a WP fault is triggered, causing a trap into the secure monitor. The secure monitor then conducts a security check based on a predefined policy (Section 3.2.3). In addition to page-table control, operations that can access the MMU must be exclusively controlled by the secure monitor. These operations encompass modifications to the page-table and the paging mechanism. We eliminate all instances of these operations from the deprived OS, including both aligned and misaligned instructions. This ensures that the deprived OS can neither alter the MMU states nor circumvent the secure monitor. Moreover, the injection of new

Table 2

Physical page attributes.

Attribute	Description
Owner	The entity to which the physical page belongs.
Type	The type of memory that the physical page is.
Max_use	The max number of times the physical page can be mapped.
Used	The number of times the physical page has been mapped.
Perm	Access control restrictions on the physical page.

instructions into the depriveleged OS could pose a threat to the secure monitor's security. However, since the secure monitor intervenes in all page-table update operations, it can effectively prevent instruction injection and data execution, thereby maintaining code integrity. For performance optimization, we map most of the secure monitor code (excluding sensitive instructions) as executable in both the secure monitor and non-TCB components.

Our system significantly reduces the TCB of the virtualization layer by restricting the control of MMU to the secure monitor. On this basis, the secure monitor provides a series of security services for the isolation of HClave.

3.2.2. Security services

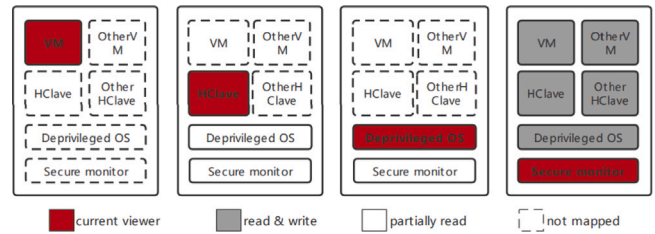
The secure monitor furnishes fundamental support for HClave's isolation by offering three key security services: memory protection, instruction protection, and protection during context switching.

Memory Protection is the first category of security services. Given that each HClave possesses its own private data, it is imperative for the secure monitor to safeguard this information. The secure monitor manages the memory layout of the entire system by controlling the MMU. It provides secure memory allocation, memory access control, and memory update control, all crucial for the protection of private data.

Secure memory allocation. The secure monitor is equipped with an internal allocator responsible for allocating the protected data. Additionally, the secure monitor allocates an internal memory pool to log the attributes of physical pages for each allocated or free memory unit. The attributes, detailed in Table 2, are utilized to control memory access and updates through relevant policies (Section 3.2.3).

Memory access control. Fig. 2 provides a visual representation of the memory access view for each component within our system. For instance, the left-most large box represents the memory access view of the VM, signifying that each VM has unrestricted access to its own memory but is denied access to others. The secure monitor, having access to all memory, dictates the memory view of each component by managing the memory mapping. Any unauthorized memory accesses across components will trigger page faults, which are subsequently detected by the secure monitor.

Memory update control. This protection mechanism enforces updates to memory mapping and sensitive data. As previously mentioned, the secure monitor is the sole component capable of modifying the page tables to control memory mapping. For instance, when the depriveleged OS seeks to update its page table, it must explicitly enter the secure monitor via a secure call gate. The secure monitor then determines the validity of this update based on the attribute information in the memory pool. Specifically, the secure monitor first assesses the level of the page-table-page. If the current page is the last level page, the secure monitor traverses the memory pool to ascertain whether the physical page has been assigned. The update request is deemed valid only if this page has been assigned and its owner is the depriveleged OS. If the current page is not the last level page, the secure monitor checks whether the system has enabled the huge page mechanism. The update request is considered valid only if the huge page mechanism is enabled and the owner of the physical page is the depriveleged OS. Additionally, the secure monitor checks specific bits in page table entries to ensure that this operation does not conflict with the system

**Fig. 2.** Memory access view of VM, HClave, depriveleged OS, and secure monitor.

protection mechanism (i.e., WP). Regarding sensitive data updates, all sensitive data are categorized into static and dynamic data. Static data, once allocated, can never be modified, while modifications to dynamic data are subject to checks by the secure monitor through its page attributes (Table 2).

Instruction protection is the second category of security service. In addition to memory protection, some special instructions also need to be taken seriously by the secure monitor. Such instructions can be classified into two categories: protection-related ones and control flow-related ones. The former is utilized to enable or disable protection mechanisms, while the latter could potentially hijack the control flow. We categorize these as sensitive instructions and prohibit their direct execution outside the secure monitor. Furthermore, the secure monitor imposes restrictions on sensitive instructions using methods such as instruction check, instruction unmap, and instruction hard-coded.

Instruction check means that the secure monitor does a sanity check with policy (Section 3.2.3) after executing the sensitive instructions. Specifically, the secure monitor checks if specific bits in some registers (e.g., CR0.WP) representing the security mechanisms are still set.

Instruction unmap is more restrictive than *instruction check*, used to enforce control flow-related instructions. It utilizes a dynamic instruction mapping approach to map instructions before execution and unmap them after execution. Specifically, the secure monitor allocates one internal physical page (4 KB) for each instruction and ensures each of them has only one instance on the physical page. Before mapping an instruction page, the secure monitor checks the protection mechanisms and parameters to prevent a compromised component from directly accessing these instructions. The secure monitor then maps the page to the component. After executing instructions, the secure monitor does instruction checks and unmaps their pages. Since the secure monitor controls all memory mapping updates, any code injection attack to bypass the protection mechanism from a compromised component will be detected and stopped.

Instruction hard-coded forces the entry point of all calls to instructions in the non-TCB components to be hard-coded. The locations to invoke these instructions are built-in during the system bootup. A binary scanner is used to make sure that there are no such instructions in unexpected code regions.

Context-switching protection is the last category of security service. When an HClave instance or the depriveleged OS needs to update its memory mapping, modify sensitive data, or execute sensitive instructions normally, it must switch to the secure monitor first. The secure monitor provides a **secure gate** for these switches. In this secure gate, the secure monitor first disables interrupts to prevent unexpected control flow transfers and then disables the CPU write-protection mechanism. In addition, the secure monitor also switches to a secure stack for execution. After the secure monitor processes these operations, it enables the write-protection mechanism before returning to the ordinary contexts. This secure gate maintains the following rules to make context switches between components secure. First, all addresses that call this gate are explicitly hard-coded in non-TCB components and loaded during the system bootup so that all non-TCB components can only enter the secure monitor through this gate. Second, interrupts are disabled to ensure that switches are atomic and cannot be redirected to other non-TCB components. Fig. 3 illustrates the control flow of our system.

Table 3
Sensitive instructions and policy enforcement.

Type	Instruction	Consequence	Policy
Protection-related	MOV CR0	Threaten the paging and the write-protection mechanisms.	Prohibit clearing the PG and WP bits of CR0.
	MOV CR4	Threaten the supervisor-mode execution/access prevention.	Prohibit clearing the SMEP/SMAP bits of CR4.
	WRMSR	Threaten the No-execution protection.	Prohibit clearing the NXE bit of EFER.
Control flow-related	MOV CR3	Tamper with memory address space.	CR3 value must be a declared valid pagetable.
	MOV CR8	Threaten the external interrupts priority.	Prohibit modifying the TPL-related bits of CR8.
	VMRUN	Change the control flow to run the virtual machines.	Prohibit modifying specific VMCS regions.
	VMRESUME	Change the control flow to resume the virtual machines.	Prohibit modifying specific VMCS regions.
	VMLAUNCH	Change the control flow to resume the virtual machines.	Prohibit modifying specific VMCS regions.
	VMOFF	Turn off the VMX mode of the CPU to destroy the system.	The execution of this instruction is prohibited.

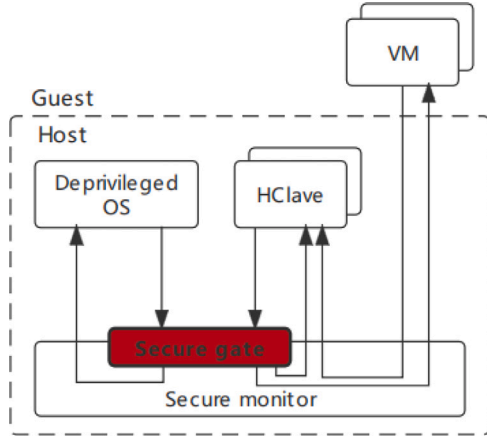


Fig. 3. The control flow between components.

3.2.3. Policy enforcement

The secure monitor stipulates some policies to enforce instruction and memory protection.

As previously stated, sensitive instructions are divided into protection-related ones and control flow-related ones. Both types of instructions are monopolized by the secure monitor. Each instruction has only one instance in the secure monitor's code regions. Based on this, the secure monitor establishes stringent policies for their execution. Details specific to the x86 architecture are presented in Table 3. The secure monitor conducts a sanity check in accordance with these policies, ensuring that these instructions are not misused.

Memory protection is one of the core security services in the secure monitor. There are four levels of policies for memory protection, corresponding to the increasing security requirements:

(P1): Page can be read and written by its owner non-TCB component. It is mapped as read-only in other non-TCB components.

(P2): Page can be read and written by its owner non-TCB component. It is unmapped in other non-TCB components.

(P3): Page is mapped as read-only in its owner non-TCB component. It is unmapped in other non-TCB components.

(P4): Page is unmapped in all non-TCB components.

It is worth noting that P4 is only used to protect the confidential data of the secure monitor. In addition, all code pages are write-protected, and all data pages are non-executable.

3.3. Isolation mechanism of HClave

3.3.1. Isolation between the deprivileged OS and HClave

Preventing the deprivileged OS from attacking HClave is the first goal of our system. Generally, the deprivileged OS code runs at a high privilege level and controls all physical memory with the kernel master pagetable. A compromised deprivileged OS can arbitrarily access HClaves' private memory. We isolate the deprivileged OS from HClaves

by creating an isolated address space for each HClave instance and restricting the deprivileged OS' access to HClaves.

Through having its own pagetable, each HClave instance has a specific address space, which is separated from the deprivileged OS. All pagetables are maintained by the secure monitor and cannot be accessed by the deprivileged OS. Memory areas are mapped differently in the deprivileged OS and HClave instances so that the former cannot access the private memory areas of the latter. Specifically, when allocating private data memory for an HClave instance, the secure monitor maps these memory areas to this HClave's address space and unmaps them from the master pagetable. Memory mapping of an HClave instance is initialized during its creation, and the mapping updates are controlled by the secure monitor through *secure memory allocation* and *memory update control*. Because the secure monitor controls all MMU operations, the deprivileged OS cannot violate the confidentiality or integrity of HClave instances. In addition, an HClave instance can specify the security policy for its private data when requesting memory allocation. We classify the data segments of HClave into six types, and each type is protected by a specific policy as shown in Table 4.

Sensitive instructions (Table 3) may directly or indirectly affect the protection mechanism for HClave instances. The secure monitor prohibits direct execution of these instructions in the deprivileged OS. All instances of such instructions in the deprivileged OS are removed by the secure monitor. Executing such instructions must be monitored and tracked by the secure monitor. Each instruction has only one instance in the secure monitor's code region. For protection-related instructions, the secure monitor restricts them through *instruction check*. Specifically, the secure monitor checks if specific bits in some registers representing the security mechanisms are still set. Because the secure monitor code is also mapped as executable in the deprivileged OS, these instructions are visible to the deprivileged OS. Therefore, a compromised deprivileged OS may disable the protection mechanisms by calling the secure monitor's interface or jumping to these instructions directly. Through *instruction check*, even if the deprivileged OS clears the protection bit, the secure monitor can perceive and fix it immediately before a bad result occurs. Control flow-related instructions are restricted by the secure monitor using *instruction unmap*. The secure monitor unmaps these instructions from the deprivileged OS and remaps them before execution. Each instruction instance is allocated on a separate 4K-page, and the secure monitor checks parameters before remapping it. For example, the secure monitor checks whether the parameter is a valid address before mapping MOV CR3.

As shown in Fig. 3, when a VM exit occurs, the secure monitor catches this event and transfers the VM exit to the associated HClave instance. Usually, the HClave instance directly handles this VM exit event and returns to the guest VM through the secure monitor. Sometimes, however, the HClave instance needs to access physical resources or rely on functionalities in the deprivileged OS, so switching to the deprivileged OS context is necessary in these scenarios. HClave first calls the secure monitor's interface to trap into the secure monitor, which checks the VM exit reason and transfers this event to the deprivileged OS. After the deprivileged OS finishes processing, it calls the secure gate to return to the secure monitor. The secure monitor then verifies the results and switches back to the target HClave. Since

Table 4
HClave data types and protection policies.

Date Type	Flag	Description	Policy
HClave Normal Data	HC_RW	These data pages belong to an HClave instance. They are readable and writable to their owner HClave instance, but invisible to other ones and the depriveleged OS. An HClave instance can modify these pages without notifying the secure monitor.	P2
HClave Critical Data	HC_RW_CHECK	These data pages contain secret information to manage an HClave instance. They are readable to their owner HClave instance, but not accessible by other ones and the depriveleged OS. Writing to pages from one HClave needs to be checked by the secure monitor.	P3
Pages storing HClave Page Table	HC_RO	These pages contain the pagetable information of an HClave instance. They are mapped as read-only in HClave but invisible to the other ones and the depriveleged OS. Only the secure monitor can write these pages.	P3
Pages storing EPT Page Table	HC_RO	These pages contain the pagetable content of the VM served by the current HClave. They are mapped as read-only in its owner HClave but invisible to the others and the depriveleged OS. Only the secure monitor can write these pages.	P3
Guest-Related Data	HC_RW_CHECK	These data pages contain VM-related data and belong to the per-VM HClave. Writing to these pages from the HClave needs to be checked by the secure monitor based on the data updates policy.	P3
Monitor-granted Data	HC_RO_GRANT	These data pages belong to the secure monitor. Permissions are dynamically granted to the HClave.	P3

the secure monitor controls memory access and instruction execution, the depriveleged OS can neither access an HClave's memory without authorization nor execute sensitive instructions to bypass the protection mechanism.

3.3.2. Isolation among HClave instances

Using untrusted VM states as input is another major security threat for the hypervisor. The design of traditional hypervisors allows a malicious VM to harm other VMs by attacking the hypervisor. To defend against such an attack, we use the secure monitor to provide a robust isolation mechanism between different HClave instances.

One HClave has full access to its own memory, but no access to others. In addition, some memory areas of the secure monitor and the depriveleged OS are mapped as read-only into the HClave instance, so the HClave instance is only allowed to access those areas, but cannot modify them. The memory access view of an HClave instance is determined during its creation and should remain monitored by the secure monitor using *memory access control* and *memory update control* during its life cycle. The secure monitor maps all page-table-pages of the HClave as read-only and enables the write protection to forbid the HClave instance from updating its memory mappings. Instead, the memory mapping updates of the HClave instance need to explicitly call the secure monitor's interface. We further enforce $W \oplus X$ in the HClave's address space to ensure that no page in the HClave is both writable and executable. The secure monitor controls HClave's access to different data memory according to the policies detailed in Table 4.

The execution of sensitive instructions may hijack the control flow or break the protection mechanism. In order to prevent a compromised HClave from attacking others, the secure monitor restricts the HClave from directly accessing these instructions. Access to these instructions needs to be forwarded to the secure monitor. The secure monitor uses *instruction hard-coded* to force no these instructions to appear in unexpected code regions. In addition, the secure monitor maps the guest memory as non-executable in the HClave, ensuring that an attacker cannot generate new code using the guest memory. User-level code is prohibited from being executed at the kernel-level.

4. Implementation

4.1. HClave interfaces

We abstract an HClave IEE to encapsulate the hypervisor running environment for each VM. The secure monitor provides three types of interfaces that are used to securely manage HClaves during their life cycles. All interfaces are pre-built in the secure monitor and loaded at the system bootup stage. An HClave must explicitly call interfaces during its life cycle.

The first type is used to create or destroy HClave instances. Such interfaces include `hc_create()` and `hc_destroy()`. `hc_create()` uses the VM descriptor as a parameter, and returns an integer as the HClave handle. This handle can be used by the secure monitor to uniquely locate an HClave instance. `hc_create()` is called during VM booting stages, while `hc_destroy()` is called when the guest VM shuts down.

The second type is context transition, used to enter or exit the HClave. Such interfaces include `hc_enter()` and `hc_exit()`. When a VM exit event occurs, the CPU first traps into the secure monitor, which invokes `hc_enter()` to enter the target HClave instance. Moreover, when an HClave needs to call functions of the depriveleged OS or the secure monitor, it calls `hc_exit()` to switch to the secure monitor. According to the exit reason, the secure monitor processes this request, or forwards it to the depriveleged OS.

The third type is used to allocate or release HClave's memory regions. Such interfaces include `hc_malloc_internal_memory()` and `hc_free()`. The secure monitor invokes `hc_malloc_internal_memory()` to allocate page-aligned memory regions for HClave's private data. When allocating memory regions, a flag parameter is used to specify the data type and protection policy. `hc_free()` zeroes the HClave's memory regions before freeing them.

4.2. Deployment on x86 platform

4.2.1. Exception capture

We limit the depriveleged OS access to HClave through memory access control and paging mechanisms. In the depriveleged OS, the

master pagetable responsible for managing physical memory is called `swapper_pg_dir`. To protect HClave's memory, We map `swapper_pg_dir` pages as read-only in the deprivileged OS. HClave also enforces write-protection through setting `PTE.RW=0` and `CR0.WP=1`. If the deprivileged OS wants to update the `swapper_pg_dir`, it must explicitly trap to the secure monitor. Any unexpected update of `swapper_pg_dir` will cause a WP fault, which is caught by the secure monitor. The secure monitor has an allocator for HClave's private data. When a block of physical memory is allocated to an HClave as private data memory, the secure monitor removes its mappings from `swapper_pg_dir`. If the deprivileged OS or other host application tries to access HClave's private data, the system will generate a PF exception. The secure monitor immediately detects it and defends against this attack. It is worth noting that if there are no relevant memory mappings in the `swapper_pg_dir`, the deprivileged OS will generate an OOP which causes a system hang. To keep our system's practicability, we added a hook function instead of OOP to handle these exceptions, which generates a warning based on the current context and page fault address.

4.2.2. Switches between the deprivileged OS and HClave

There are two scenarios where HClave needs to switch to the deprivileged OS. The first is to handle EPT violation. EPT mechanism is based on the original CR3 pagetable address mapping, and furthermore, it introduces an EPT pagetable to implement another mapping. The original pagetable is responsible for guest virtual address (GVA) to guest physical address (GPA) mapping, while the EPT pagetable handles the mapping from GPA to host physical address (HPA). For each VM, in addition to the EPT pagetable, there is also a host pagetable, which is used by Qemu process where the guest VM is located. When a GPA-to-HPA does not exist in EPT, the VM generates an EPT violation, which switches CPU privilege to run KVM. KVM first translates GPA to host virtual address (HVA) through memory slots, and then searches HVA-to-HPA mapping in the host pagetable. If the HVA-to-HPA does not exist, the deprivileged OS handles it as a host page fault with `swapper_pg_dir`. In our experiments, when handling EPT violation, the HClave first traps into the secure monitor to switch to the deprivileged OS. If the HVA-to-HPA mapping exists in the host pagetable, the secure monitor notifies HClave to update the EPT pagetable using this HPA. If the mapping does not exist, a PF exception occurs. The secure monitor captures this exception and handles it using `swapper_pg_dir`.

The second is to handle device I/O. KVM uses Qemu to support I/O virtualization. Typically, a guest VM generates an I/O VM exit and enters the KVM. KVM then determines operation type and passes it to Qemu. For emulated devices, HClave uses the trap-and-emulate approach to handle both port I/O and MMIO. HClave configures the I/O field in VMCS to trap I/O operation to the secure monitor. In addition, The secure monitor controls the guest EPT and clears the mappings of virtual device I/O memory. The secure monitor captures I/O VM exit events and passes only the I/O data to Qemu. After the emulation is completed, the secure monitor checks the results and returns to the HClave. For virtio devices, HClave uses an end-to-end I/O encryption (Li et al., 2019) for virtio operation. For secure DMA, the secure monitor controls the IOMMU to track PCI devices' physical pages using paging attributes. IOMMU pagetables are protected by the secure monitor through write-protection. Any modification to IOMMU page-table-pages will cause a WP fault. In this way, HClave ensures that the host handles I/O operations correctly.

4.2.3. Intervention between HClave and VM

We enable bidirectional monitoring by interfering with the control flows between VM and HClave. When a guest is running and a VM exit occurs, the control flow is first transferred to the secure monitor. The secure monitor checks the VM exit reason and then sends this event to the specific HClave. Before sending to the HClave, the secure monitor checks the guest state regions in VMCS according to the VM exit reason,

Table 5
ARM registers in EL2.

Registers	Description
<code>hcr_el2</code>	Hypervisor configuration register.
<code>vtbtr_el2</code>	Hypervisor virtualization translation base table register.
<code>tcr_el2</code>	Hypervisor translation control register.
<code>vtcr_el2</code>	Hypervisor virtualization translation control register.
<code>hstr_el2</code>	Hypervisor system trap register.
<code>ttbr0_el2</code>	Hypervisor translation base table register.
<code>far_el2</code>	Hypervisor fault address register.
<code>hpfar_el2</code>	Hypervisor IPA fault address register.
<code>vbar_el2</code>	Hypervisor exception vector base table register.
<code>sctlr_el2</code>	System control register.
<code>cptr_el2</code>	Architectural feature trap register.
<code>esr_el2</code>	Exception syndrome register.

ensuring that no invalid parameters are passed to the HClave. If the HClave needs to access the VM's data, it will call the secure monitor to check the permissions. The memory areas, that are related to VMCS in HClave, are marked as `HC_RW_CHECK` during allocation. The secure monitor checks VMCS to ensure that they have not been tampered with before returning to the guest. For image security, the VM image is stored on the disk in an encrypted manner. Before each startup, the secure monitor performs integrity checks on the VM images and decrypts the images. The key can be managed by the cloud providers' key management service (KMS) (Amazon Web Services, 2020a).

4.3. Extension on ARM64 platform

HClave, while implemented on the Intel X86_64 platform, is not reliant on specific hardware-secure features such as Intel SGX or AMD SEV. This allows HClave to be integrated into a universal isolated execution environment template and subsequently ported to various hardware platforms. To further demonstrate the versatility of HClave's design, we have developed an additional prototype on the ARM platform.

ARM introduces the concept of Exception Level (EL) to define the CPU running context. The ARM architecture includes four ELs: user-mode EL0, kernel-mode EL1, hyp-mode EL2, and root-mode EL3. Virtualization support in ARM is achieved through Virtualization Exception (VE) (ARM, 2011), with the hypervisor operating in hyp-mode EL2. For enhanced performance, ARM has introduced Virtualization Host Exceptions (VHE), enabling the entire host OS to run on hyp-mode EL2.

Specifically, we extend HClave to include instruction protection and memory protection support for the ARM64v8 architecture. For memory protection, we employ the same memory mapping management and memory access policies as the x86 architecture. When it comes to instruction protection, there are notable differences between the x86 and ARM platforms. As previously mentioned, ARM VE introduces a new EL2 privilege level to operate the hypervisor and comes with a fresh set of registers and memory spaces. Consequently, we limit the execution and access of registers and associated instructions. These instructions can be categorized into two groups: protection-related and control flow-related. For protection-related instructions, we eliminate all instances of instructions that write to sensitive registers (Table 5) from the host OS and verify the value of the registers after each access.

For control flow-related instructions, particularly HVC, SMC, and ERET, these instructions can also only be utilized in a secure monitor. Specifically, for HVC, this instruction is usually used for communication and coordination between the hypervisor and the guest OS. Due to device limitations, our development board uses ARM64v8.0 processor, which does not support VHE. The host OS (as deprivileged OS) also runs on EL1 like guest OS and enters the HClave instance by calling the HVC. Therefore, to prevent malicious host OS from bypassing the secure monitor by calling HVC and directly entering HClave instances, which

may directly endanger the running environment of HClave instances, we remove all HVC instances from the host OS. For SMC instruction, this instruction is used by OS to interact with EL3 secure firmware. As we all know, EL3 is the highest exception level in ARM architecture and can access any physical memory or registers. A compromised OS or HClave instance may use the SMC to bypass our isolation mechanism through EL3. So we remove all SMC instances from the host OS and HClave instances. For ERET instruction, this instruction is used to return from the exception handler to the normal execution process. It is usually executed at the end of the exception handling program, switching the processor state from exception mode back to normal mode. Unlike X86, ARM uses ERET for all privilege level returns. A compromised host OS or HClave instance may use this instruction directly to return the VM and bypass the secure monitor. So we also remove all ERET instances from the host OS and HClave instances.

Compared to the deployment on the X86 platform, we implement exception-handling functions in the host OS on the ARM platform. Furthermore, we put the exception vector table and the distribution function of the exception entry into TCB to ensure that all exceptions can be assigned to the correct handling function. All code segments are set as non-writable to prevent the host OS from tampering with functions.

5. Evaluation results

5.1. Security evaluation

5.1.1. Attack vectors

Tampering with secure monitor code. During the system boot phase, we use the secure boot to ensure the integrity of the secure monitor. After booting, the secure monitor measures the integrity of the system's code and sets all code to be write-protected. In addition, the secure monitor prohibits mapping new executable memory to the deprived OS or HClave, so it can defend against code injection attacks.

Bypass the secure monitor protection. The protection mechanism could be bypassed if the sensitive instructions in Table 2 were executed directly by non-TCB components. Our system dynamically checks and unmaps those instructions in non-TCB components. For a new instruction injection attack, we defend against it by write-protection. Regarding ROP-based attacks, we check the result before a bad consequence occurs. In addition, a binary scanner is used to ensure that there are no such instructions (reuse of ROP gadget) in unexpected code regions. Hence, there is no way to bypass the secure monitor protection.

DMA attack. An attacker may try to access sensitive memory regions by leveraging DMA. We defend against such attacks by controlling IOMMU, which is used for DMA I/O address translation. The secure monitor controls the mapping updates and information in IOMMU and strictly constrains the scope that each guest VM can access.

5.1.2. Case study of practical attacks

We evaluate HClave's effectiveness against a compromised deprived OS or malicious VMs by analyzing CVEs and identifying the cases where HClave secures hypervisor runtime despite any compromise. We analyzed 22 CVEs related to Linux/KVM, and the results are shown in Tables 6 and 7. The CVEs consider two cases: a malicious VM that exploits KVM functions, and a compromised deprived OS that exploits bugs in Linux/KVM. The CVEs related to our thread model could result in privilege escalation, info leakage, memory corruption, and DoS in Linux/KVM.

Privilege escalation This type of attack is an intermediate state, leading to arbitrary attack consequences. Attackers may exploit vulnerabilities in the hypervisor or host applications to gain host OS privilege. For these attacks, our focus is to prevent further damage to other VMs after privilege escalation. In our system, the deprived OS and

HClaves are out of the TCB. Once they try to access memory beyond its permissions, it will be captured by the secure monitor immediately. So even if they are compromised, they cannot cause devastating results. Such as cve-2015-7504, cve-2013-0311, and cve-2016-9777, for these cases, we emulated an attacker that tries to access other HClave's memory by adding a function in HClave instance to access other HClave's page table. All of them triggered a page fault and were caught by the secure monitor.

Info leakage This type of attack is aimed at reading the memory of sensitive data from any part of the system. These attacks may be due to improper parameter handling or uninitialized memory. In our system, each non-TCB component is limited to its address space and isolated from others. A compromised non-TCB component cannot read any information except its own. Such as cve-2016-3713, cve-2013-1798, and cve-2015-5165, for these cases, we added a kernel module to simulate an attacker who attempts to access other HClave's normal data (e.g., kvm_arch_vcpu structure data) or VM's memory. All of these attacks failed in HClave.

Memory corruption This type of attack usually begins with out-of-bound access or mishandling of memory mapping. In our system, similar to leakage information, memory allocation, and sensitive data updates are controlled by the secure monitor. So once an HClave tries to perform an invalid memory access, the secure monitor will immediately find it and kill the VM. Such as cve-2017-2596, cve-2015-4036, and cve-2014-3601, for these cases, we deployed the same method as info case studies of info leakage. And HClave defends against these attacks.

Denial-of-Service from guest This type of attack is aimed at exploiting vulnerabilities in HClave to crash other VMs or the entire hypervisor. For attacks that try to crash other VMs, HClave can check the memory and registers of guest VM states before returning to the guest. Any mishandling state will be fixed immediately. For attacks that try to crash the hypervisor, HClave ensures that they can only affect their own instance by isolating each HClave. Such as cve-2017-1000407 and cve-2017-1000252, both of them only crashed their own HClave.

5.2. Performance evaluation

We evaluate the performance overhead of HClave both on Intel X86 and ARM64 platforms by running various benchmarks in the guest OS, e.g., SPEC CPU2017 (SPEC, 2017) or Unixbench (Lucas, 2016) and kernel compilation to evaluate the macrobenchmark overhead, as well as IOzone (Capps and Norcott, 2019) and DD (Torvalds, 2021) to evaluate I/O overhead. We also test five real-world application workloads (shown in Table 8) to evaluate the performance of HClave in the real environment. In addition, we further measure some microbenchmarks to further understand the factors that affect performance. Our experiments have two deployments: (1) an X86 server with Intel Core i7-7700K processor and 16 GB physical memory, running Ubuntu 18.04.5 with Linux kernel 5.2.0 as host OS (deprived OS), assigning four virtual CPU cores and 4 GB memory to one guest VM, which runs Ubuntu 18.04.5 with Linux kernel 5.4.0. (2) an ARM64v8 development board with 2-core Cortex-A72 64-bit and 4-core Cortex-A53 64-bit, 4 GB physical memory, running firefly-rk3399 as host os, assigning one virtual CPU core and 2 GB memory to one guest VM, which runs ubuntu-server 18.04 with Linux kernel 5.4.0. In terms of system configuration, for the X86 platform, we tested native KVM, HClave, and nested KVM; for ARM64 platform, we tested native KVM and HClave. Due to the performance limitations of the ARM64 development board, we did not test the performance of nested KVM on the ARM platform. According to the NEVE (Lim et al., 2017), the performance overhead of nested virtualization on ARM may still require more optimization.

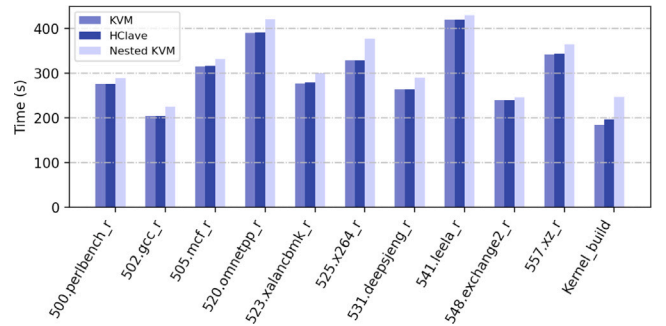
Table 6

Case study: from the deprived OS.

CVE-*	Description	Defense	KVM	HClave
2019-13272	Privilege escalation: Mishandles the recording of the credentials of a process that wants to create a ptrace relationship.	An attacker obtains root permission by leveraging certain scenarios with a parent-child process relationship. Then, we emulated the deep attack to access HClave's pagetable. As expected, it triggered pagefault.	No	Yes
2016-5195	Privilege escalation: Improper handling of a copy-on-write (COW) feature to write to a read-only memory mapping.	First, an attacker gains privileges by leveraging incorrect handling of a COW feature. Then, we emulated the deep attack to access HClave's pagetable. As expected, it triggered pagefault.	No	Yes
2009-3234	Privilege escalation: Host kernel buffer overflow in kernel/perf_counter.c.	First, an attacker executes arbitrary code via a "big size data" to the perf_counter_open system call. Then, we emulated the deep attack to modify HClave's pagetable. As expected, it triggered pagefault.	No	Yes
2018-18021	Privilege escalation: Mishandling of VM register state allows local users to redirect kernel control flow.	An attacker can arbitrarily redirect the control flow by leveraging incorrect KVM_SET_ON_REG ioctl handler. Then, we emulated the deep attack to modify HClave's pagetable. As expected, it triggered pagefault.	No	Yes
2013-2234	Info leakage: Improperly initializing of structure members, which allows local users to obtain secrets from kernel heap memory.	An attacker obtains secrets from the kernel heap by reading a broadcast message from the notify interface of an IPSec key_socket. Since HClave has its own secure heap memory, attackers cannot achieve the final goal.	No	Yes
2016-9756	Info leakage: Improperly initializing of code segment, which allows local users to obtain sensitive information.	An attacker obtains sensitive information from kernel stack memory via a crafted application. Since HClave has its own secure stack memory, attackers cannot achieve the final goal.	No	Yes
2010-4525	Info leakage: Improperly initialize of interrupt.pad structure member, which allows local users to obtain sensitive information from kernel stack memory.	An attacker obtains potentially sensitive information from kernel stack memory via unspecified vectors. Since HClave has its own secure stack memory, attackers cannot achieve the final goal.	No	Yes
2013-4592	Denial-of-Service: Mishandling memory slots in virt/kvm/kvm_main.c allows local users to cause memory consumption.	An attacker causes a DoS (memory consumption) by leveraging certain device access to trigger the movement of memory slots. Since the secure monitor can manage the allocation of certain memory slots, some cases can be avoided.	No	Partially solved
2013-4129	Denial-of-Service: Improperly check of timer, which allows local users to cause a denial of service.	An attacker causes a denial of service (BUG and system crash) via vectors involving the shutdown of a KVM virtual machine. Since the bug directly shuts down the VM, it is out of our scope.	No	No

5.2.1. X86 platform performance

SPEC CPU and kernel build. We measure the performance of all applications in the SPEC CPU2017 benchmark in a guest VM running on the original KVM (as the baseline performance), HClave, and nested KVM. We ran nine rounds of each benchmark with workload and the average runtime of each benchmark is shown in Fig. 4. Specifically, for CPU-intensive benchmarks, such as perlbench_r, gcc_r, and x264_r, there is nearly no overhead since HClave hardly intercepts CPU execution or introduces additional VM exits. Compared with CPU-intensive benchmarks, memory-intensive benchmarks, e.g., mcf_r, and deepsjeng_r, a slight performance overhead is observed (from 0.3% to 0.7%). This is because in HClave, sensitive memory operations, especially pagetable updation and sensitive data access, are monitored and checked by the secure monitor, leading to an increased frequency of control-flow switches between HClaves and the deprived OS. However, because we control most of the checks in the secure monitor, and according to Tables 2 and 4, most of the checks are register and memory page property checks, even in the worst cases, i.e., the kernel compilation, the overhead is only 6.47%, which is much smaller than nested KVM (33.39%). Overall, the average overhead is 0.8%.

**Fig. 4.** SPEC CPU performance on X86 platform.

IOzone and DD. To evaluate the disk I/O overhead, we select IOzone benchmarks, with the settings of 4 KB block size and 32 MB file size. We first try 100 rounds of the benchmarks, but find the standard deviations of the throughput for each benchmark is extremely large. Then we run 1000 rounds of each benchmark, compute the average

Table 7

Case study: from the VM.

CVE-*	Description	Defense	KVM	HClave
2015-7504	Privilege escalation: Heap-based buffer overflow in hw/net/pcnet.c allows guest OS administrators to execute arbitrary code.	First, an attacker executes arbitrary code via a series of packets in loopback mode. Then we emulated the deep attack to access other HClave's pagetable. As expected, it triggered page fault.	No	Yes
2013-0311	Privilege escalation: Improperly handling cross-region descriptors in drivers/vhost/vhost.c allows guest OS to obtain host OS privilege.	First, an attacker obtains host OS privileges by leveraging KVM guest OS privileges. Then, we emulated the deep attack to modify HClave's pagetable. As expected, it triggered page fault.	No	Yes
2016-9777	Privilege escalation: Improperly restricting the VCPU index allows guest OS users to gain host OS privileges.	First, an attacker gains host OS privileges via a crafted interrupt request. Then, we emulated the deep attack to modify HClave's pagetable. As expected, it triggered page fault.	No	Yes
2018-1087	Privilege escalation: Improper handling of exception, which allows guest users to escalate their privileges to guest OS.	An attacker escalates the privilege in the guest by using the flaw in handling exceptions. Since the bug only influences the attacker's own VM, we regard this attack as an inner attack and it is out of our scope.	No	No
2016-3713	Info leakage: Improperly handling MSR $0 \times 2f8$ in arch/x86/kvm/mtrr.c allows guest OS users to obtain sensitive information.	An attacker obtains sensitive information via a crafted ioctl call. We emulated the attack to read or write to the other HClave's kvm_arch_vcpu data structure. As expected, it triggered page fault.	No	Yes
2013-1798	Info leakage: Improperly handling operations in virt/kvm/ioapic.c allows guest OS users to obtain sensitive information from the host.	An attacker obtains sensitive information from host OS memory via a crafted application. Since secure data is isolated from host OS memory, an attacker cannot achieve the final goal.	No	Yes
2015-5165	Info leakage: Improperly RTL8139 emulation allows guest OS to read up to 65 KB of QEMU heap memory.	An attacker reads heap memory via unspecified vectors. Since HClave has its own secure heap memory, an attacker cannot achieve the final goal.	No	Yes
2017-2596	Memory corruption: Improperly emulating the VMXON instruction in arch/x86/kvm/vmx.c allows guest OS users to cause host OS memory corruption.	An attacker can cause host OS memory consumption by leveraging the mishandling of page references in the traditional design. But in our system, the VMXON will be trapped into the secure monitor, this attack can be prevented.	No	Yes
2015-4036	Memory corruption: Array index error in the tcm_vhost_make_tpg function allows guest OS users to cause memory corruption.	An attacker causes memory corruption via a crafted VHOST_SCSI_SET_ENDPOINT ioctl call. In our system, the attacker only influences its own HClave instance.	No	Yes
2014-3601	Memory corruption: Mishandling page mapping in virt/kvm/iommu.c allows guest OS users to cause host OS memory corruption.	An attacker can cause memory corruption by triggering an improper gfn value. In our system, the scope of the gfn is controlled by the secure monitor, therefore the memory corruption can be prevented.	No	Yes
2017-1000407	Denial-of-Service: Flooding the I/O port to crash the hypervisor.	An attacker can cause a kernel panic by flooding the diagnostic port 0×80 . In our system, the attacker only influence its own HClave, because the HClaves provide services to their own guest independently.	No	Yes
2017-1000252	Denial-of-Service: Out-of-bounds value causes hypervisor crash.	An attacker can crash the hypervisor via an out-of-bounds guest_irq value. In our system, the attacker only influence its own HClave, because the HClaves provide services to their own guest independently.	No	Yes
2014-7842	Denial-of-Service: KVM bug allows guest users to crash their own virtual machine.	An attacker can crash its guest VM via a crafted application. We regard this attack as an inner attack and it is out of our scope.	No	No

throughput, and show the results in Fig. 5. Overall, the average I/O performance overhead of HClave is 5% (nested KVM is 10%). For some test cases (e.g., stride-read, record-rewrite, and fwrite) however, the performance of HClave is better than that of the native KVM. We believe the reason is caching and buffering, since the frequent memory operations of HClave cause some I/O data to be accessed from memory instead of disk. Some test results of nested KVM are also for the same reason. We further use DD to evaluate I/O performance, with the

configuration of 128 MB block size, 1 GB file size, and direct mode. The read and write overheads are about 0.8% and 6.5% respectively, as shown in Table 9.

Application workloads. We then conduct the evaluation on five real-world applications: a web server Apache, a scheduler benchmark suite Hackbench, a memory benchmark suite Memcached, a network I/O benchmark suite Netperf and a database benchmark suite Sysbench. Table 8 lists the configuration of these benchmarks on X86 platform.

Table 8
Application benchmarks.

Name	X86 Platform Configuration	ARM64 Platform Configuration
Apache	Apache Web Server v2.4.41 (The Apache Software Foundation, 2023) runs inside a VM and Apache Bench v2.3 (The Apache Software Foundation, 2015) runs inside the host, simulating 20 concurrent clients to request test pages of different sizes from 1 KB to 128 KB with 1GB data per-page.	Apache server v2.4.58 (The Apache Software Foundation, 2023) runs inside a VM and handling 100 concurrent requests from remote Apache Bench v2.3 (The Apache Software Foundation, 2015), serving from 4 KB to 64 KB pages with 1GB data per-page.
Hackbench	Hackbench (Ubuntu, 2019), included in rt-tests v1.5-2, runs in process mode with 50 groups using 1000 file descriptors each. Each sender will pass 500 messages from 1 KB to 128 KB.	Hackbench (Russell, 2024) included in LTP-20240129, runs in process mode with 25 groups, 40 pairs of sender and receiver with 100 datasize.
Memcached	Memcached v1.5.22(RedisLabs, 2022) uses the Memtier Benchmark v1.3.0 (Redis, 2023) with memcache_text protocol and data from 1 KB to 128 KB. Each size of data repeats 100 times.	Memcached v1.6.22(RedisLabs, 2023) using memtier benchmark v2.0.0 (Redis, 2023) with default configuration, serving from 16Byte to 256Byte data.
Nerperf	Netperf v2.6.0(Hewlett Packard Enterprise, 2005) runs a server inside a VM and a client inside the host, testing TCP_STREAM, TCP_CRR, TCP_RR, UDP_STREAM and UDP_RR with 100 MB and 20 rounds.	Netperf v2.7(Hewlett Packard Enterprise, 2005) runs a server inside a VM and a client inside the host, testing TCP_STREAM, TCP_CRR, TCP_RR, UDP_STREAM and UDP_RR with 100 MB and 20 rounds.
Sysbench (MySQL)	MySQL v8.0.30 (Oracle, 2023) runs inside a VM and Sysbench v1.0.18(Kopytov, 2023) runs inside the host, using six cases including read_write, delete, insert, point_select, update_index and update_non_index, running 10 threads for 100 s, 100 tables with 1000 rows per-table.	MySQL v8.2.0 (Oracle, 2023) handling requests from sysbench benchmarks v1.0.2 (Kopytov, 2023), running workload using six types of operations (the same as the X86 platform) with 5 tables, 1000 counts per-table, 10 threads, and lasting for 30 s.

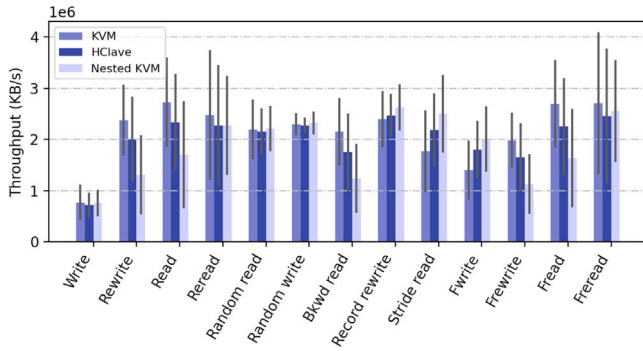


Fig. 5. I/O performance on X86 platform.

Table 9
Disk I/O performance overhead using DD on X86 platform.

Operation	KVM	HClave	Nested KVM
128MB-READ (MB/s)	182.50	180.95 (0.8%)	108.05 (40.8%)
128MB-WRITE (MB/s)	83.35	77.95 (6.5%)	75.2 (9.8%)

The results of Apache are shown in [Fig. 6\(a\)](#), as file size increases, the throughput tends to be saturated and stable. The average overhead introduced by HClave is 8%. For Hackbench, as shown in [Fig. 6\(b\)](#), the performance overhead is from 9% to 18%. The main factor causing this performance overhead is that HClave needs to invoke the scheduler frequently due to multiple instances. The performance overhead of Memcached is about 7%. For Nerperf and Sysbench, we normalize the results so that a value of 1.0 means the same performance as the native KVM. Higher numbers mean less overhead. The overhead of them introduced by HClave is 1% and 6% respectively. Totally, the overhead for HClave on the X86 platform in many cases is less than 10%, which is much better than the Nested virtualization (more than 30%).

CPU cycles. We further conduct some microbenchmarks to better understand the factors that cause the overhead. These tests are run

100,000 times to get the average values. We first test the number of CPU cycles added by switching to the different HClave. The results show that the overhead is 2626 cycles. Then we test the number of CPU cycles added by switching to the TCB module. The overhead is 235 cycles.

TCB complexity. HClave introduces only 2249 lines of C and 87 lines of assembly code as TCB (measured by cloc [Al Danial, 2020](#)). We remove the traditional OS, hypervisor, and each HClave instance out of the TCB, so as long as the TCB of our system is verified to be secure, there will be no vulnerability for control-flow hijacks or ROP attacks. According to [Gu et al. \(2016\)](#), current verification can be applied to more than 10K LOC, so it is reasonable to verify HClave's correctness, which is left as our future work.

5.2.2. ARM64 platform performance

Unixbench and kernel build. We measure the performance of all applications in the Unixbench benchmark in a guest VM running on the original ARM/KVM (as the baseline) and HClave. Similar to X86, we also ran nine rounds of each benchmark with workload and the average runtime of each benchmark is shown in [Fig. 7](#). Specifically, for most CPU-intensive benchmarks, e.g., Whetstone, Dhrystone, and System call, HClave hardly introduces performance overhead (less than 1%). This is because HClave hardly intercepts control flow (such as frequent context switching) and interferes with CPU operation. For memory-intensive benchmarks, there is a modest performance overhead of approximately 2%~4%, which is because HClave involves permission checks when accessing sensitive memory. Overall, the average performance overhead of HClave on ARM64 platform is 3.8%. For the kernel build, we use linux-4.19.309 with allnoconfig. The overhead of HClave is about 12.8%, as shown in [Table 10](#).

IOzone and DD. We select IOzone and DD as the I/O benchmarks on the ARM64 platform. The configuration of IOzone is 4 KB block size with 32 MB file size. We also ran 1000 rounds of each benchmark and computed the average. The result is shown in [Fig. 8](#). Compared to x86, the ARM platform exhibits poorer IO performance, with an average overhead of around 10%. Furthermore, we also use DD to evaluate I/O

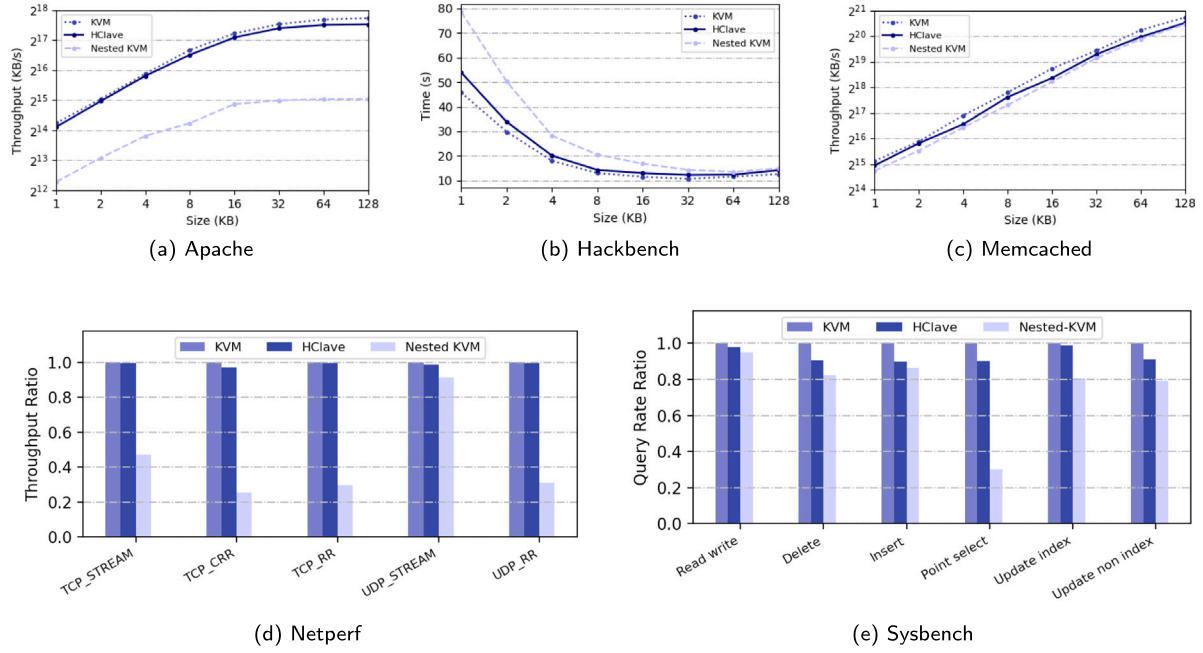


Fig. 6. Application workloads on X86 platform.

Table 10
Kernel build on ARM64 platform.

Operation	KVM on ARM64	HClave on ARM64
Kernel build (s)	415.33	468.62

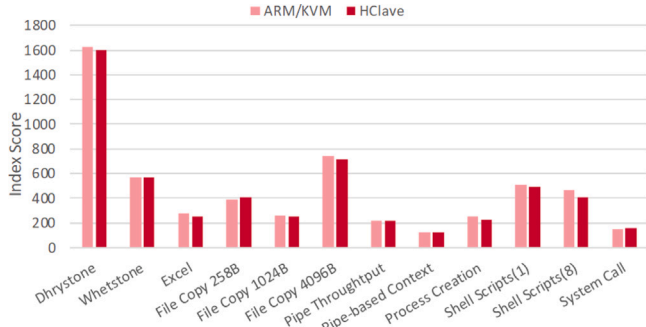


Fig. 7. Unixbench performance on ARM64 platform.

Table 11
Disk I/O performance overhead using DD on ARM64 platform.

Operation	KVM on ARM64	HClave on ARM64
128MB-READ (MB/s)	115.10	77.108 (33%)
128MB-WRITE (MB/s)	15.14	12.43 (17.9%)

performance on ARM64, with the configuration of 128 MB block size, 1 GB file size, and direct mode. The average overhead is about 25%, as shown in Table 11. In general, the IO performance on ARM64 platform is inferior to that of X86 platform.

Application workloads. We also conducted evaluations on five real-world applications: Apache, Hackbench, Memcached, Netperf, and MySQL (Sysbench). The configuration details of these benchmarks on the ARM64 platform are presented in Table 8. Regarding the Apache benchmark, we measured the processing time for each request. The results, as depicted in Fig. 9(a), indicate that HClave incurs an approximate performance overhead of 1% as the size of requested pages

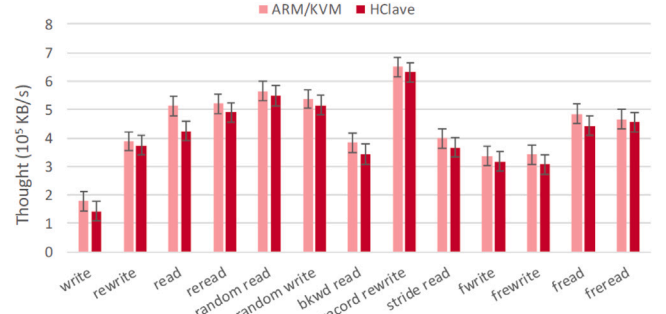


Fig. 8. I/O performance on ARM64 platform.

increased. For the Netperf benchmark, the overhead observed is approximately 1.7%. In the case of the Hackbench benchmark, HClave exhibits an overhead of around 10%. In the Memcached and Sysbench benchmarks, we observe relatively higher performance overhead. The average overhead values are approximately 5% and 11% respectively. The increased overhead can be attributed to the frequent memory operations involved in both benchmarks, leading to frequent permission checks and control-flow switches in HClave. In summary, the average overhead of HClave on the ARM64 platform is approximately 5.4%.

TCB complexity. HClave introduces only 2087 lines of C code and 87 lines of assembly code (measured by CLOC [Al Danial, 2020](#)) as the TCB of the normal world on ARM64 platform.

6. Discussion and limitation

Side-channel attacks. Side-channel attack and defense have become one of the research hotspots in the field of system security. Although side-channel attacks, e.g., [Lipp et al. \(2018\)](#), [Kocher et al. \(2019\)](#), are not considered in our threat model, HClave does not conflict with the existing side-channel defenses. Since HClave does not depend on specific hardware-secure mechanism, it is compatible with hardware defense solutions directly. HClave is also orthogonal to software defenses like [Chartre and Marsden, 2019, 2018](#), so they can be incorporated easily into HClave.

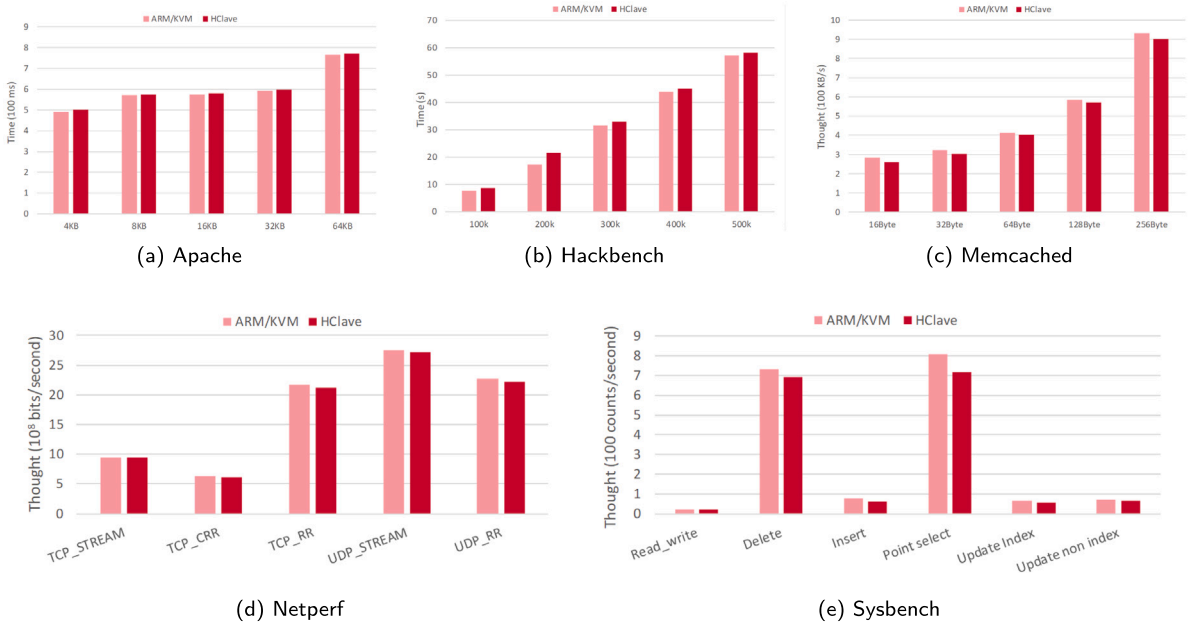


Fig. 9. Application workloads on ARM64 platform.

DoS attacks. There are two scenarios for DoS attacks. On the one hand, DoS attacks can be launched against the guest OS by malicious applications inside the guest OS. Although our system was not designed to defeat such attacks, the secure monitor can help mitigate such attacks by interfering with the current context of the VM. On the other hand, DoS attacks can also be launched from the deprived OS, which is a limitation of the current design.

Sensitive instruction construction attacks. Although we use a binary scanning program to ensure that there are no sensitive instructions in non-TCB regions, attackers may form sensitive instructions in other ways. Take an arithmetic instruction “add eax, 0x300f” as an example. This instruction actually includes a hidden WRMSR τ (0x0f30) instruction. We may not remove this arithmetic instruction since it is related to its functionality. Therefore, the attacker may manipulate ip register point to the middle of this arithmetic instruction to form the WRMSR sensitive instruction.

Template design problems. In our system, all HClave instances share the same template. As long as an attacker exploits a vulnerability of HClave to attack one instance, all instances may be compromised.

7. Related work

Hypervisor decoupling. Decoupling hypervisors (Wang et al., 2012; Wu et al., 2013; Shi et al., 2017; Zhang et al., 2011; Mi et al., 2020; Colp et al., 2011; Shi et al., 2012; Amazon Web Services, 2020b; Microsoft, 2022, 2016) to increase the security of virtualization layers has been studied for many years. HyperLock and DeHype assign separate hypervisor instances to each VM, isolating them. HClave also creates isolated contexts for VMs but focuses on protecting the hypervisor even if the host OS is compromised. MultiHype supports multiple hypervisors on one platform, while HClave uses a per-VM HClave instance to achieve stronger isolation. Cloudvisor (Zhang et al., 2011) and Cloudvisor-D (Mi et al., 2020) employ nested virtualization to safeguard VMs from a compromised hypervisor. Unlike HClave, these systems concentrate on isolating the effects of the hypervisor on VMs rather than fortifying the hypervisor itself. Nexen (Shi et al., 2017) and HClave both employ Nested Kernel to reconstruct the virtualization platform, but Nexen does not the security vulnerabilities within its shared service domain. AWS Nitro Enclave (Amazon Web Services, 2020b), which is based on the host OS and KVM, includes the host OS in its TCB. Managed

Windows Defender Credential Guard (Microsoft, 2022) and Microsoft Hyper-V Shielded VM (Microsoft, 2016) are designed to protect sensitive data within Hyper-V VMs. Compared to these works, HClave tends to provide bidirectional protection between VMs and hypervisor instances, rather than just protecting VMs.

Secure hardware. Numerous studies (Microsoft, 2022; Jin et al., 2015; Xia et al., 2013; Strackx and Piessens, 2012; Intel, 2020; T. Alves, 2004; Li et al., 2022; Kim et al., 2017) have introduced secure hardware to protect VMs. H-SVM (Jin et al., 2015) moves memory management from the hypervisor to Hardware (SMM) and denies the hypervisor’s access to protected memory by rejecting page mapping requests for the corresponding protected pages. ARM CCA (Li et al., 2022) and Intel TDX (Intel, 2020) have created trusted execution environments to ensure the security of VMs. Compared to these works, HClave aims to achieve better cross-platform universality without relying on specific secure hardware environments. Furthermore, the design of HClave can also be considered to be combined with the hardware confidential computing environment (e.g., ARM CCA) to further enhance the security of virtualization platforms.

8. Conclusion

This paper introduces a new software IEE that can enhance hypervisor runtime security and achieve better portability. We protect the control-flow and data-flow of the whole virtualization layer through three security services: memory protection, instruction protection, and context-switching protection. We implemented two prototypes of our design on X86 and ARM platforms respectively and evaluated their security and practicability. The experimental results show that our system can defeat the exploits against the malicious system software and VMs with reasonable performance overhead.

CRedit authorship contribution statement

Qihang Zhou: Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Wenzhuo Cao:** Software. **Xiaoqi Jia:** Funding acquisition. **Shengzhi Zhang:** Writing – review & editing, Methodology. **Jiayun Chen:** Software, Data curation. **Nan Jiang:** Software. **Weijuan Zhang:** Resources. **Haichao Du:** Resources. **Zhenyu Song:** Funding acquisition. **Qingjia Huang:** Resources.

Declaration of competing interest

No potential conflict of interest was report by the authors.

Data availability

The data that has been used is confidential.

Acknowledgments

We would like to thank the anonymous reviewers for their constructive comments. This work was supported in part by Program of Key Laboratory of Network, Assessment Technology, the Chinese Academy of Sciences. Program of Beijing Key Laboratory of Network Security and Protection Technology. National Key Research and Development Program of China, No. 2021YFB2910109. National Natural Science Foundation of China (NO. 62202465).

References

- Al Danial, 2020. cloc. <https://github.com/AlDanial/cloc>.
- Amazon Web Services, 2020a. AWS key management service. <https://aws.amazon.com>.
- Amazon Web Services, 2020b. AWS nitro system. <https://docs.aws.amazon.com/enclaves/>.
- ARM, 2011. ARMv8 white paper. <https://community.arm.com>.
- Azab, A.M., Ning, P., Shah, J., Chen, Q., 2014. Hypervision across worlds: Real-time kernel protection from the ARM TrustZone secure world. In: ACM CCS. pp. 90–102.
- Baumann, A., Peinado, M., Hunt, G.C., 2014. Shielding applications from an untrusted cloud with haven. In: OSDI. pp. 267–283.
- Ben-Yehuda, M., Day, M.D., Dubitzky, Z., Fa, M., 2010. The turtles project: Design and implementation of nested virtualization. In: OSDI. pp. 423–436.
- Capps, D., Norcott, W., 2019. Iozone. <http://iozone.org/>.
- Chartre, A., Marsden, G., 2018. Kernel page-table isolation. https://en.wikipedia.org/wiki/Kernel_page-table_isolation.
- Chartre, A., Marsden, G., 2019. Improve security with address space isolation. <https://blogs.oracle.com/linux/improve-security-with-address-space-isolation-asi>.
- Colp, P., Nanavati, M., Zhu, J., Aiello, W., Coker, G., Deegan, T., Loscocco, P., Warfield, A., 2011. Breaking up is hard to do: security and functionality in a commodity hypervisor. In: SOSP. pp. 189–202.
- CVE, 2021. CVE. <http://cve.mitre.org>.
- D, N., K, T., Dietz, W., 2015. Nested kernel: An operating system architecture for intra-kernel privilege separation. Computer architecture news.
- Gu, R., Shao, Z., Chen, H., Wu, X., Kim, J., Sjöberg, V., Costanzo, D., 2016. Certikos: An extensible architecture for building certified concurrent OS kernels. In: USENIX Symposium on Operating Systems Design and Implementation. pp. 653–669, URL <https://api.semanticscholar.org/CorpusID:1796034>.
- Hewlett Packard Enterprise, 2005. Netperf. <https://hewlettpackard.github.io/netperf/>.
- Hofmann, O.S., Kim, S., Dunn, A.M., Lee, M.Z., Witchel, E., 2013. InkTag: secure applications on an untrusted operating system. In: ASPLOS. pp. 265–278.
- Intel, 2020. Intel trust domain extensions. <https://software.intel.com>.
- Intel Corporation, 2016. Intel software guard extensions programming reference. <https://software.intel.com>.
- Jin, S., Ahn, J., Seol, J., Cha, S., Huh, J., Maeng, S., 2015. H-SVM: Hardware-assisted secure virtual machines under a vulnerable hypervisor. IEEE Trans. Comput.
- Kaplan, D., Powell, J., Woller, T., 2016. AMD memory encryption. <https://developer.amd.com/>.
- Kim, K., Lee, H., Kang, B., Byunghoon, H., Ingoo, P., 2017. Detecting and preventing kernel rootkit attacks with bus snooping. IEEE Trans. Dependable Secure Comput.
- Kivity, A., Kamay, Y., Laor, D., L., U., L., A., 2007. KVM: the linux virtual machine monitor. In: Ottawa Linux Symposium.
- Kocher, P., Horn, J., Fogh, A., Genkin, D., 2019. Spectre attacks: Exploiting speculative execution. In: S&P. pp. 1–19.
- Kopytov, A., 2023. Sysbench. <https://github.com/akopytov/sysbench>.
- Li, S., Koh, J.S., Nieh, J., 2019. Protecting cloud virtual machines from commodity hypervisor and host operating system exploits. In: USENIX Security. pp. 1357–1374.
- Li, X., Li, X., Dall, C., Gu, R., Nieh, J., Sait, Y., Stockwell, G., 2022. Design and verification of the arm confidential compute architecture. In: 16th USENIX Symposium on Operating Systems Design and Implementation. OSDI 2022, Carlsbad, CA, USA, July 11–13, 2022, pp. 465–484.
- Lim, J.T., Dall, C., Li, S.W., Nieh, J., Zyngier, M., 2017. NEVE: Nested virtualization extensions for ARM. In: Proceedings of the 26th Symposium on Operating Systems Principles. pp. 201–217. <http://dx.doi.org/10.1145/3132747.3132754>.
- Lipp, M., Schwarz, M., Gruss, D., 2018. Meltdown: Reading kernel memory from user space. In: USENIX Security. pp. 973–990.
- Lucas, K., 2016. UnixBench. <https://github.com/kdlucas/byte-unixbench>.
- Mi, Z., Li, D., Chen, H., Zang, B., Guan, H., 2020. (Mostly) exitless VM protection from untrusted VMM through disaggregated nested virtualization. In: USENIX Security. pp. 1695–1712.
- Microsoft, 2016. Microsoft hyper-V shielded VM. <https://www.techtarget.com/searchwindowsserver/definition/Microsoft-Hyper-V-Shielded-VM>.
- Microsoft, 2022. Manage windows defender credential guard.
- Oracle, 2023. MySQL. https://hub.docker.com/_/mysql.
- Redis, 2023. Memtier benchmarks. https://github.com/RedisLabs/memtier_benchmark.
- RedisLabs, 2022. Memcached. https://github.com/RedisLabs/memtier_benchmark.
- RedisLabs, 2023. Memcached. https://hub.docker.com/_/memcached.
- Russell, R., 2024. Linux test project. <https://github.com/linux-test-project/ltp/releases/tag/20240129>.
- Shi, W., Lee, J., Suh, T., Woo, D.H., Zhang, X., 2012. Architectural support of multiple hypervisors over single platform for enhancing cloud computing security. In: CF. pp. 75–84.
- Shi, L., Wu, Y., Xia, Y., Da, N., C, H., Zang, B., Li, J., 2017. Deconstructing xen. In: NDSS.
- Shih, M., Kumar, M., Kim, T., Gavrilovska, A., 2016. S-NFV: securing NFV states by using SGX. In: SDN Workshop.
- SPEC, 2017. <https://www.spec.org/cpu2017/>.
- Strackx, R., Piessens, F., 2012. Fides: selectively hardening software application components against kernel-level or process-level malware. In: ACM CCS.
- T. Alves, D.F., 2004. Trustzone: Integrated hardware and software security. ARM white paper.
- Tboot, 2021. Tboot. <https://sourceforge.net/projects/tboot/>.
- The Apache Software Foundation, 2015. ab-The Apache Software Foundation. <https://httpd.apache.org/docs/2.4/programs/ab.html>.
- The Apache Software Foundation, 2023. httpd. <https://hub.docker.com/layers/library/httpd/latest/images>.
- Torvalds, 2021. Device driver. <https://github.com/torvalds/linux>.
- Ubuntu, 2019. Hackbench. <https://manpages.ubuntu.com/manpages/xenialman8/hackbench.8.html>.
- Wang, Z., Wu, C., Grace, M.C., Jiang, X., 2012. Isolating commodity hosted hypervisors with HyperLock. In: EuroSys. pp. 127–140.
- Wu, C., Wang, Z., Jiang, X., 2013. Taming hosted hypervisors with (mostly) deprived execution. In: NDSS.
- Wu, Y., Yutao, L., Ruifeng, L., Chen, H., Binyu, Z., 2018. Comprehensive VM protection against untrusted hypervisor through retrofitted AMD memory encryption. In: HPCA. pp. 441–453.
- Xia, Y., Hua, Z., Yu, Y., 2022. Colony: A privileged trusted execution environment with extensibility. IEEE Trans. Comput. 71 (2), 479–492.
- Xia, Y., Liu, Y., Chen, H., 2013. Architecture support for guest-transparent VM protection from untrusted hypervisor and physical attacks. In: HPCA. pp. 246–257.
- Zhang, F., Chen, J., Chen, H., Zang, B., 2011. CloudVisor: retrofitting protection of virtual machines in multi-tenant cloud with nested virtualization. In: SOSP. pp. 203–216.
- Zhou, Q., Jia, X., Zhang, S., Jiang, N., Chen, J., Zhang, W., 2022. SecFortress: Securing hypervisor using cross-layer isolation. In: 2022 IEEE International Parallel and Distributed Processing Symposium. IPDPS 2022, Lyon, France, May 30 - June 3, 2022, IEEE, pp. 212–222.

Qihang Zhou received the Ph.D degree in Cyberspace Security from University of Chinese Academy of Sciences, Beijing, China, in June 2022. She is working as a postdoctor in the Institute of information engineering, Chinese Academy of Sciences, Beijing, China. Her research interests include system security, virtualization security and confidential computing.

Wenzhuo Cao is a doctoral student of Chinese Academy of Sciences, Beijing, China. His research interests include confidential computing, system security.

Xiaoqi Jia received the Ph.D degree in Information Security from the Graduate School of the Chinese Academy of Sciences, Beijing, China, in 2010. He is currently a professor with Institute of information engineering, Chinese Academy of Sciences, China. His research interests include cloud security, AI security and malicious code analysis.

Shengzhi Zhang received the Ph.D. degree from the Department of Computer Science and Engineering, Penn State University, in 2012. He was an Assistant Professor with the Florida Institute of Technology, and a Researcher with the IBM Research Laboratory, Honeywell Aerospace, and Cisco Research and Development. He is currently an Associate Professor with the Computer Science Department, Metropolitan College, Boston University. His research interests include AI security, the IoT security, system security, vehicle security, and mobile security.

Jiayun Chen is a doctoral student of Chinese Academy of Sciences, Beijing, China. His research interests include trusted computing, virtualization security.

Nan Jiang is a doctoral student of Chinese Academy of Sciences, Beijing, China. His research interests include trusted execution environment, mobile security.

Weijuan Zhang received the Ph.D degree in Information Security from the Institute of Information Engineering, Chinese Academy of Sciences, China. She is working as a Senior engineer in the Institute of information engineering, Chinese Academy of Sciences, Beijing, China. Her research interests include cloud computing security.

Haichao Du received the master's degree in Software engineering from Central South University, Changsha, China, in June 2015. She is working as a Senior engineer in the Institute of information engineering, Chinese Academy of Sciences, Beijing, China. Her research interests include system security, malicious code detection, network security.

Zhenyu Song received the Ph.D degree in Electronics and Communications Engineering from Polytechnic University of Turin, Turin, Italy, in November 2013. He is working as a professor in the Institute of information engineering, Chinese Academy of Sciences, Beijing, China. His research interests include system security and cloud computing security.

Qingjia Huang received the Ph.D degree in Computer Science and Technology from Beijing University of Posts and Telecommunications, Beijing, China, in July 2014. He is working as a Senior Engineer in the Institute of information engineering, Chinese Academy of Sciences, Beijing, China. His research interests include cloud computing security, advanced threat detection and malicious code analysis.